Design Document
Command-line-based Monopoly Game

PolyU COMP3211 Software Engineering Project, Fall 2024

Group 24

November 22, 2024

1 Architecture Design

1.1 Architecture Overview

The Model-View-Controller (MVC) architecture is adopted in the design of the Monopoly game. The source code of the system is divided into three parts: Model, View, and Controller. The MVC architecture is chosen for several reasons and offers numerous advantages in this context:

- **Separation of Concerns**: By dividing the application into three interconnected components, MVC allows developers to focus on one aspect of implementation at a time. This separation makes the codebase more manageable and easier to understand.
- **Modularity**: Each component (Model, View, Controller) can be developed, tested, and maintained independently. This modularity facilitates easier debugging and enhances the ability to extend the system with new features.
- **Reusability**: The Model component, which contains the core game logic and data, can be reused across different types of views (e.g., graphical user interface, command-line interface) without modification.
- **Scalability**: MVC architecture supports scalability by allowing the addition of new views or controllers without affecting the existing system. This is particularly useful for games like Monopoly, which may evolve over time with new rules or interfaces.
- **Maintainability**: The clear separation between the Model, View, and Controller makes it easier to locate and fix bugs, as well as to update the system. Changes in the user interface or business logic can be made independently.
- **Testability**: MVC promotes better testability by allowing unit tests to be written for each component independently. This ensures that the game logic can be thoroughly tested without the need for a user interface.

Classes in the Model package define the basic data types and relevant logic for data manipulation, including the important elements of the game, such as `Game`, `Player`, `Property`, `EffectType`, and `Effect`. The Model is responsible for maintaining the state of the game and implementing the core game mechanics. It ensures that the game rules are enforced and provides methods for querying and updating the game state.

The Controller package is mainly responsible for the initialization, execution, saving, loading, continuation, and termination of the game. It acts as an intermediary between the Model and the View, processing user inputs, invoking model updates, and updating the view accordingly. The Controller also handles file-intensive operations such as saving and loading game states, ensuring that the game can be paused and resumed seamlessly.

The View package is responsible for the presentation layer of the game. It gathers the classes that define the data structures for implementing visual elements (`GameBoard`, `Box`, and `Brick`), and

functions for some important system prompts (`Info`). The View constructs the user interface and displays the current state of the game to the player. It also captures user interactions and forwards them to the Controller for processing.

For simplicity, some interfacing logic regarding data visualization is implemented directly in the Model package, and some user modifications and events are directly handled in the View package before being sent to the Model package. However, the Controller package is responsible for most of the coordination between the Model and View packages and the overall control of the game. This separation of concerns ensures that the system is modular, making it easier to maintain and extend in the future.

1.2 Architecture Diagram

The diagram illustrating the architecture of the Monopoly game is shown in Figure 1.

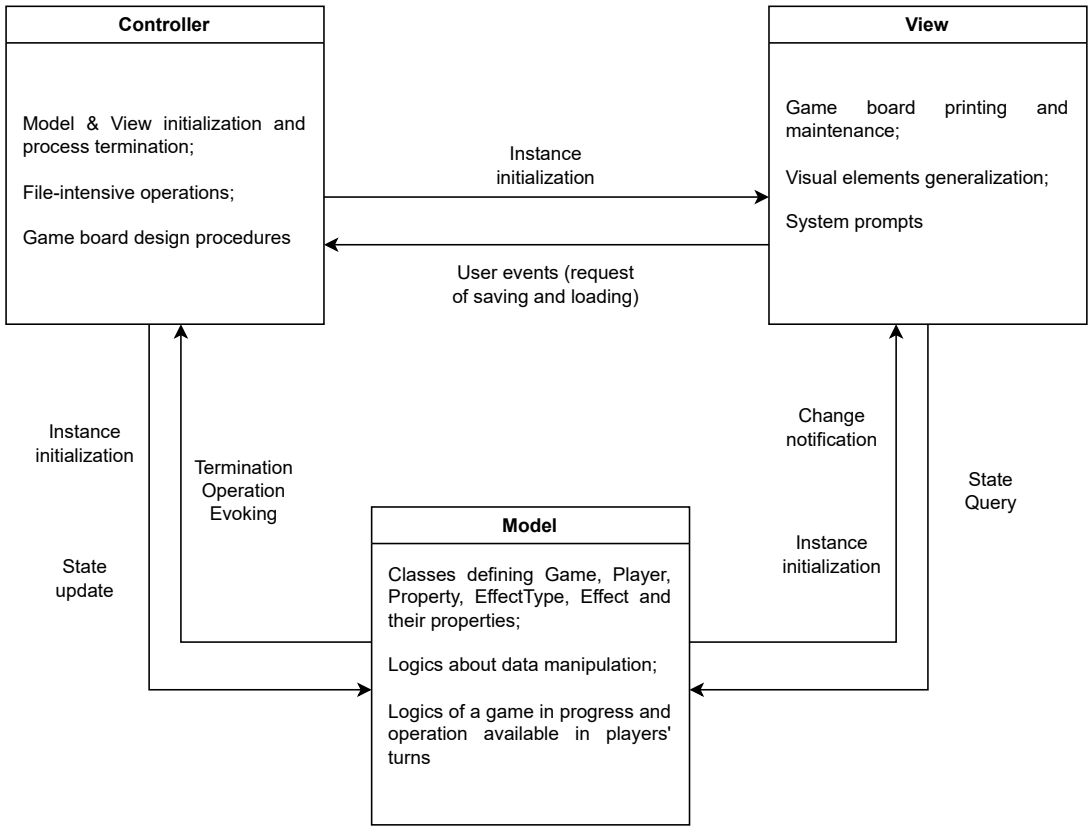


Figure 1: Architecture Diagram

2 The Structure and the Relationship among Components

- The structure and the relationship among main code components are illustrated in the class diagram on the penultimate page.
- The overall use cases for player and designer are shown in Figure 2 and Figure 3 respectively.

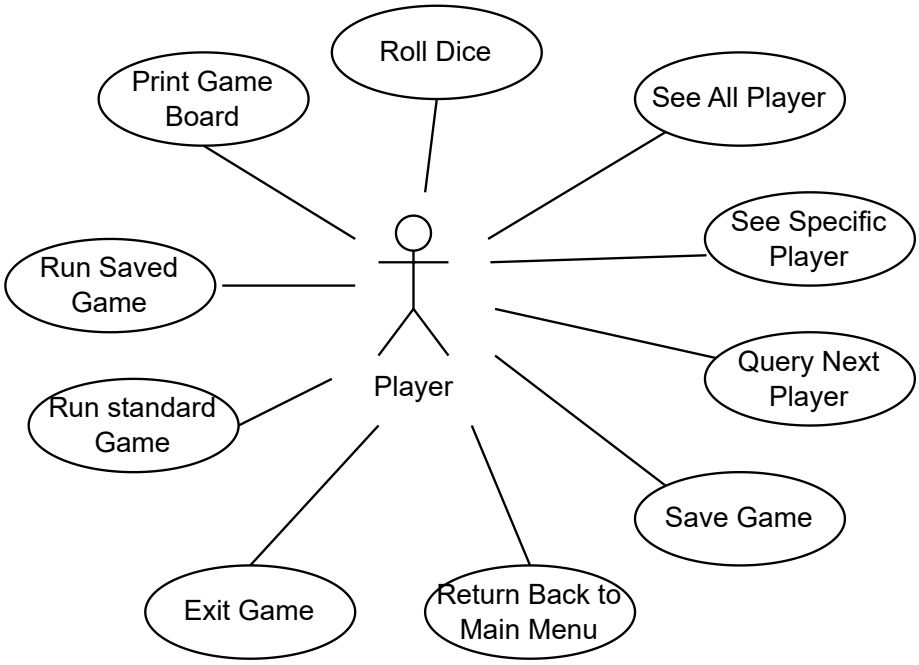


Figure 2: Use Cases Diagram for Player

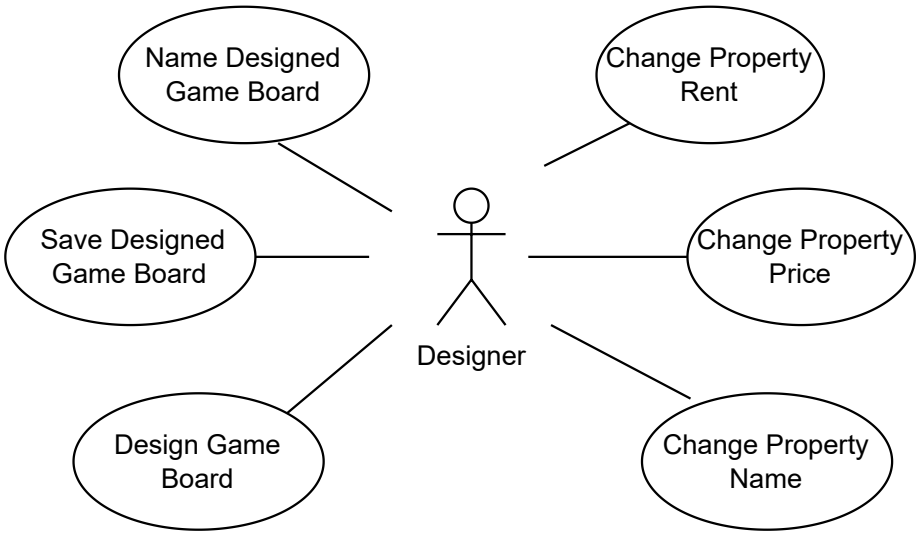


Figure 3: Use Cases Diagram for Designer

3 Example Use during a Turn

The example use outlining the flow during a user's turn of playing the game is attached in on the last page. Below is a brief summary of the sequence of events during a player's turn for the sequence diagram.

Lifelines Game System: The overarching system that manages the game flow, including player turns and game state. Game: Represents the current game's logic, including checks for game over conditions and player actions. Player: Represents the individual player interacting with the game system. Sequence of Events

Game Loop The game begins with the startGame() function, which initializes the game state. The system enters a loop that continues as long as the game is not over ([!gameOver]).

Game State Checks The checkGame() method is called to determine if the game should continue or end. If the game is over, the flow diverges into two branches: If gameOver is false, the game continues. If gameOver is true, the GameOver method is invoked, prompting the end of the game.

Player Turn The playerTurn() method is called to manage the actions of the current player. If the player is in jail (isInJail), the JailTurn() method is executed, handling the specific rules for players who cannot move freely. If the player is not in jail, the user menu is printed for the player to make a choice. The turnOver variable is used to control the flow, determining whether the player's turn has ended or continues. If turnOver is false, the loop continues, allowing the player to make further choices until they decide to end their turn.

User Menu Interaction The player is presented with options (choices) through the printUserMenu() method within a loop. Depending on the player's choice.

Choice 1: Calls the rollDiceWithPrint() method, which simulates rolling the dice and prints the result. After, triggers the positionMove(Dice, Player) method, moving the player based on the dice result. Ends the player's turn by setting turnOver=true.

Choice 2: Calls the printBoard() method and prints the current game Board. Continues the player's turn by continue setting turnOver=false and let user make further decisions on player menu.

Choice 3: Displays information about all players using the allPlayer(Info) method. Continues the player's turn by continue setting turnOver=false and let user make further decisions on player menu.

Choice 4: Allows the player to check information on a specific player via specificPlayer(Info). Continues the player's turn by continue setting turnOver=false and let user make further decisions on player menu.

Choice 5: Queries the next player's turn with queryNextPlayer(). Continues the player's turn by continue setting turnOver=false and let user make further decisions on player menu.

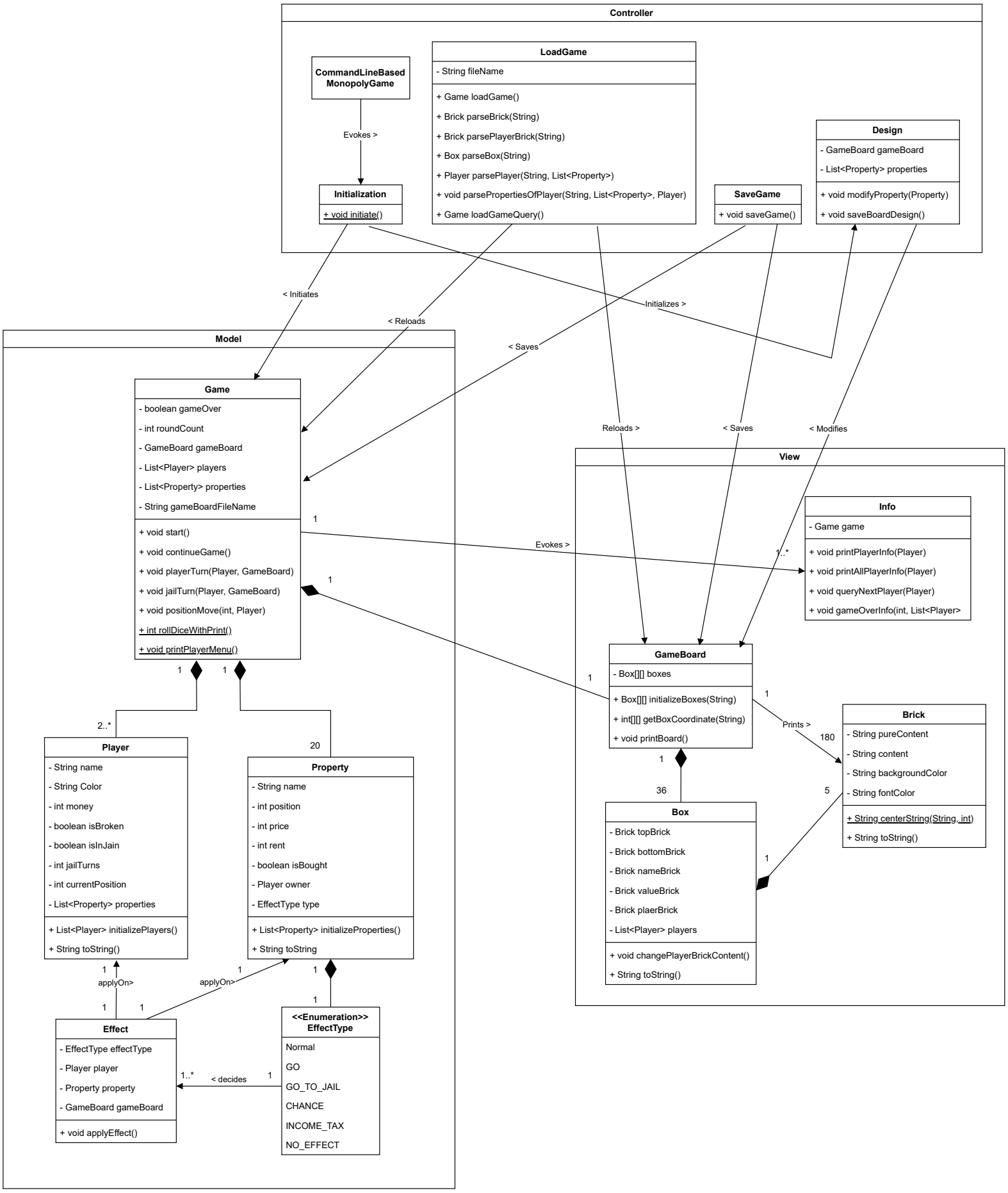
Choice 6: Saves the game state using saveGame(txt.file). Continues the player's turn by continue setting turnOver=false and let user make further decisions on player menu.

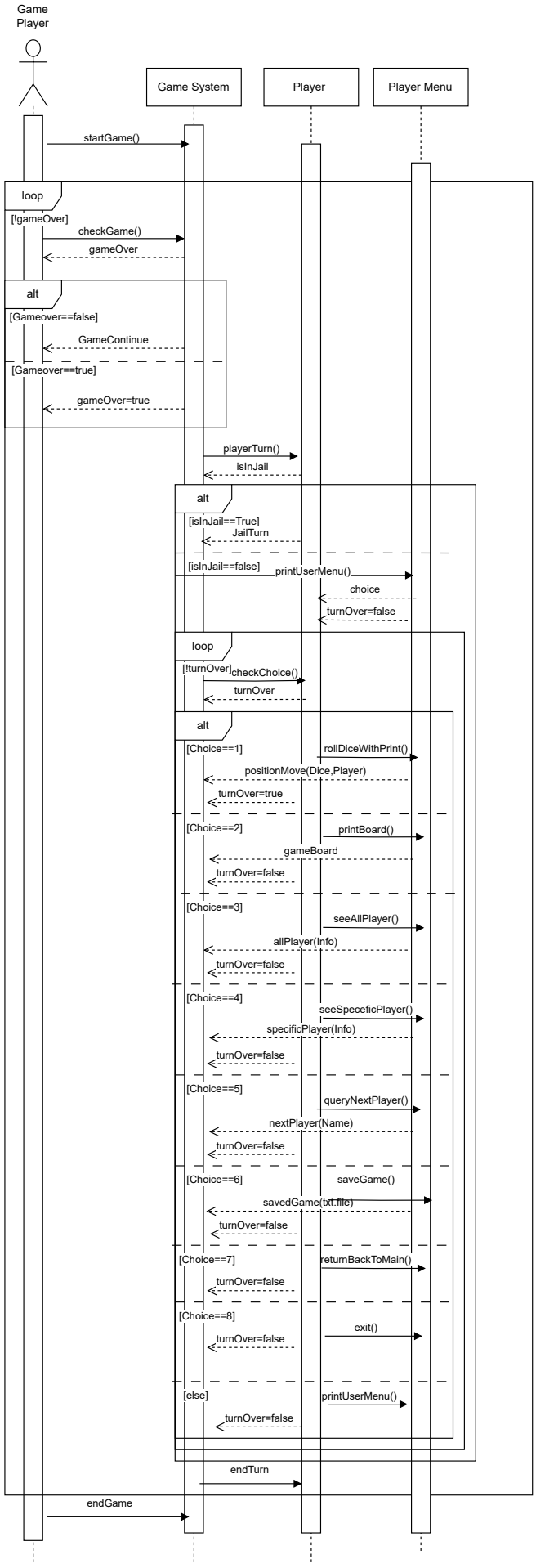
Choice 7: User chooses to return back to main menu, the returnBacktoMain() function is called, the game continues, and the turnover is set to true as the turn is over. A new game is waited to be started for the user at the main menu.

Choice 8: User chooses to exit the game and sets the turnOver=true and gameOver=true so the two loops ends, and the game is exited.

If none of the choices are entered like a 9 or letters, the game system will generate an "invalid choice, please try again" message and the call printUserMenu() method again for the player to choose.

Game Over Handling If gameOver is set to true, the game reaches a conclusion, exiting the game loop and concluding the game session.


Controller
CommandLineBased MonopolyGame
Evokes >
Initialization
+ void initiate()
LoadGame
- String fileName
+ Game loadGame()
+ Brick parseBrick(String)
+ Brick parsePlayerBrick(String)
+ Box parseBox(String)
+ Player parsePlayer(String, List<Property>)
+ void parsePropertiesOfPlayer(String, List<Property>, Player)
+ Game loadGameQuery()
SaveGame
+ void saveGame()
Design
- GameBoard gameBoard
- List<Property> properties
+ void modifyProperty(Property)
+ void saveBoardDesign()
< Initiates
< Reloads
Initializes >
< Saves
Model
Game
- boolean gameOver
- int roundCount
- GameBoard gameBoard
- List<Player> players
- List<Property> properties
- String gameBoardFileName
+ void start()
+ void continueGame()
+ void playerTurn(Player, GameBoard)
+ void jailTurn(Player, GameBoard)
+ void positionMove(int, Player)
+ int rollDiceWithPrint()
+ void printPlayerMenu()
Reloads >
< Saves
< Modifies
View
Info
- Game game
+ void printPlayerInfo(Player)
+ void printAllPlayerInfo(Player)
+ void queryNextPlayer(Player)
+ void gameOverInfo(int, List<Player>
Evokes >
GameBoard
- Box[][] boxes
+ Box[][] initializeBoxes(String)
+ int[][] getBoxCoordinate(String)
+ void printBoard()
Prints >
Brick
- String pureContent
- String content
- String backgroundColor
- String fontColor
+ String centerString(String, int)
+ String toString()
Box
- Brick topBrick
- Brick bottomBrick
- Brick nameBrick
- Brick valueBrick
- Brick plaerBrick
- List<Player> players
+ void changePlayerBrickContent()
+ String toString()
Player
- String name
- String Color
- int money
- boolean isBroken
- boolean isInJain
- int jailTurns
- int currentPosition
- List<Property> properties
+ List<Player> initializePlayers()
+ String toString()
Property
- String name
- int position
- int price
- int rent
- boolean isBought
- Player owner
- EffectType type
+ List<Property> initializeProperties()
+ String toString
applyOn>
Effect
- EffectType effectType
- Player player
- Property property
- GameBoard gameBoard
+ void applyEffect()
< decides
<<Enumeration>> EffectType
Normal
GO
GO_TO_JAIL
CHANCE
INCOME_TAX
NO_EFFECT


Game Player
Game System
Player
Player Menu
startGame()
loop
[!gameOver]
checkGame()
gameOver
alt
[Gameover==false]
GameContinue
[Gameover==true]
gameOver=true
playerTurn()
isInJail
alt
[isInJail==True]
JailTurn
[isInJail==false]
printUserMenu()
choice
turnOver=false
loop
[!turnOver]
checkChoice()
turnOver
alt
[Choice==1]
rollDiceWithPrint()
positionMove(Dice,Player)
turnOver=true
[Choice==2]
printBoard()
gameBoard
turnOver=false
[Choice==3]
seeAllPlayer()
allPlayer(Info)
turnOver=false
[Choice==4]
seeSpeceficPlayer()
specificPlayer(Info)
turnOver=false
[Choice==5]
queryNextPlayer()
nextPlayer(Name)
turnOver=false
[Choice==6]
saveGame()
savedGame(txt file)
turnOver=false
[Choice==7]
returnBackToMain()
turnOver=false
[Choice==8]
exit()
turnOver=false
[else]
printUserMenu()
turnOver=false
endTurn
endGame